

Polynomial Representation and Addition

Using Linked Lists

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Recall: Linked List Node

In a normal singly linked list, each node contains:



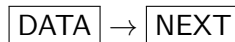
For polynomial representation, one DATA field is not enough.

A polynomial term has two important values:

- Coefficient
- Exponent

Therefore, the node must store both.

Normal node:



Polynomial node:



What is a Polynomial?

Consider:

$$P(x) = 6x^3 + 9x^2 + 7x + 1$$

Each term has two parts:

Term	Coefficient	Exponent
$6x^3$	6	3
$9x^2$	9	2
$7x$	7	1
1	1	0

$$\text{Term} = \text{Coefficient} \times x^{\text{Exponent}}$$

Representing One Term

Consider:

$$6x^3$$

Here:

$$\text{Coefficient} = 6$$

$$\text{Exponent} = 3$$



6
⏟
Coefficient

3
⏟
Exponent

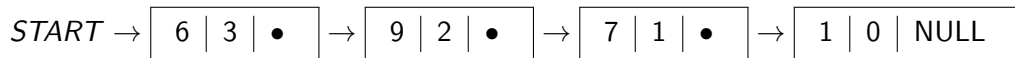
We can store the term in one linked-list node.

Linked Representation of a Polynomial

Consider:

$$P(x) = 6x^3 + 9x^2 + 7x + 1$$

Each term becomes one node:



Each node stores:

<i>Coefficient</i>		<i>Exponent</i>		<i>Next</i>
--------------------	--	-----------------	--	-------------

Polynomial Node in C

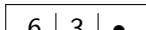
A suitable node structure is:

```
struct Node
{
    int coeff;
    int exp;
    struct Node *next;
};
```

Corresponding node:



Example:



Why Use a Linked List for Polynomials?

Consider:

Linked lists are useful because:

- Only existing polynomial terms need storage.
- Zero-coefficient terms need not be stored.
- Polynomial size can grow dynamically.
- Terms can be inserted or removed easily.
- Addition can be performed by simultaneous traversal.

$$P(x) = 8x^{100} + 5x^3 + 2$$

An array representation may require many unused positions.

Linked list stores only:

$$(8, 100) \rightarrow (5, 3) \rightarrow (2, 0)$$

How Should Terms Be Stored?

For easier addition, terms should usually be stored in descending order of exponent.

Correct ordering:

$$6x^5 + 4x^3 + 2x + 1$$

Linked list:

$$(6, 5) \rightarrow (4, 3) \rightarrow (2, 1) \rightarrow (1, 0)$$

Why?

Terms with equal powers can be matched easily

Creating a Polynomial Node

```
struct Node *createNode(int coeff, int exp)
{
    struct Node *newNode;

    newNode =
        (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed\n");
        return NULL;
    }

    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = NULL;
}
```

Adding a Term at the End

```
struct Node *addTerm(  
    struct Node *start,  
    int coeff,  
    int exp)  
{  
    struct Node *newNode;  
    struct Node *ptr;  
  
    newNode = createNode(coeff, exp);  
  
    if (start == NULL)  
        return newNode;  
  
    ptr = start;  
  
    while (ptr->next != NULL)  
        ptr = ptr->next;
```

Creating a Polynomial

```
1 struct Node *createPolynomial()
2 {
3     struct Node *start = NULL;
4     int n, coeff, exp;
5
6     printf("Enter number of terms: ");
7     scanf("%d", &n);
8
9     for (int i = 0; i < n; i++)
10    {
11        printf("Enter coefficient: ");
12        scanf("%d", &coeff);
13
14        printf("Enter exponent: ");
15        scanf("%d", &exp);
16
17        start = addTerm(start, coeff, exp);
18    }
```

Displaying a Polynomial

```
void displayPolynomial(struct Node *start)
{
    struct Node *ptr = start;

    while (ptr != NULL)
    {
        printf("%dx^%d", ptr->coeff, ptr->exp);

        if (ptr->next != NULL)
            printf(" + ");

        ptr = ptr->next;
    }

    printf("\n");
}
```

Polynomial Addition

Consider:

$$P(x) = 5x^3 + 4x^2 + 2x + 1$$

and:

$$Q(x) = 3x^3 + 2x^2 + 7$$

Addition is performed by combining terms having the same exponent.

$$5x^3 + 3x^3 = 8x^3$$

$$4x^2 + 2x^2 = 6x^2$$

Therefore:

Two Polynomials as Linked Lists

Polynomial 1

$$P(x) = 5x^3 + 4x^2 + 2x + 1$$

$$P : (5, 3) \rightarrow (4, 2) \rightarrow (2, 1) \rightarrow (1, 0)$$

Polynomial 2

$$Q(x) = 3x^3 + 2x^2 + 7$$

$$Q : (3, 3) \rightarrow (2, 2) \rightarrow (7, 0)$$

We traverse both lists simultaneously.

Two-Pointer Technique

Use:

$PTR1$

for Polynomial 1.

Use:

$PTR2$

for Polynomial 2.

At each step, compare:

with:

$PTR1 \downarrow$

$(5, 3) \rightarrow (4, 2) \rightarrow (2, 1)$

$PTR2 \downarrow$

$(3, 3) \rightarrow (2, 2) \rightarrow (7, 0)$

Three Cases During Addition

At every step, compare the exponents.

Case 1

$$PTR1 - > exp = PTR2 - > exp$$

Add coefficients.

Case 2

$$PTR1 - > exp > PTR2 - > exp$$

Copy term from Polynomial 1.

Case 3

$$PTR1 - > exp < PTR2 - > exp$$

Case 1: Equal Exponents

Suppose:

$$PTR1 \rightarrow (5, 3)$$

and:

$$PTR2 \rightarrow (3, 3)$$

Since:

$$3 = 3$$

add coefficients:

$$5 + 3 = 8$$

Result node:

$$(8, 3)$$

Case 2: First Exponent is Larger

Suppose:

$$PTR1 \rightarrow (6, 4)$$

and:

$$PTR2 \rightarrow (5, 3)$$

Since:

$$4 > 3$$

there is currently no matching x^4 term in Polynomial 2.

Therefore copy:

$(6, 4)$

into the result.

Then move on to:

Case 3: Second Exponent is Larger

Suppose:

$$PTR1 \rightarrow (5, 2)$$

and:

$$PTR2 \rightarrow (7, 3)$$

Since:

$$2 < 3$$

copy:

$$(7, 3)$$

into the result.

Then move only:

Polynomial Addition: Dry Run

Given:

$$P(x) = 5x^3 + 4x^2 + 2x + 1$$

$$Q(x) = 3x^3 + 2x^2 + 7$$

Start:

$$PTR1 \rightarrow (5, 3)$$

$$PTR2 \rightarrow (3, 3)$$

Equal exponents:

$$3 = 3$$

Dry Run: Next Terms

Now:

$$PTR1 \rightarrow (4, 2)$$

$$PTR2 \rightarrow (2, 2)$$

Equal exponents:

$$2 = 2$$

So:

$$4 + 2 = 6$$

Result becomes:

$$\boxed{8x^3 + 6x^2}$$

Dry Run: Unequal Powers

Now:

$$PTR1 \rightarrow (2, 1)$$

$$PTR2 \rightarrow (7, 0)$$

Since:

$$1 > 0$$

copy:

$$2x$$

Result:

$$8x^3 + 6x^2 + 2x$$

Dry Run: Final Terms

Now:

$$PTR1 \rightarrow (1, 0)$$

$$PTR2 \rightarrow (7, 0)$$

Equal powers:

$$0 = 0$$

Add coefficients:

$$1 + 7 = 8$$

Final result:

$$8x^3 + 6x^2 + 2x + 8$$

Addition Trace Table

PTR1	PTR2	Action	Result Term
(5, 3)	(3, 3)	Add	(8, 3)
(4, 2)	(2, 2)	Add	(6, 2)
(2, 1)	(7, 0)	Copy P1	(2, 1)
(1, 0)	(7, 0)	Add	(8, 0)

$$8x^3 + 6x^2 + 2x + 8$$

Algorithm for Polynomial Addition

- ① Set PTR1 to first polynomial.
- ② Set PTR2 to second polynomial.
- ③ Repeat while both pointers are not NULL.
- ④ If exponents are equal:
 - Add coefficients.
 - Add resulting term to output.
 - Move PTR1 and PTR2.
- ⑤ If PTR1 exponent is larger:
 - Copy PTR1 term.
 - Move PTR1.
- ⑥ Otherwise copy PTR2 term and move PTR2.
- ⑦ Copy all remaining terms from the non-empty list.

Polynomial Addition: C Code – Part 1

```
struct Node *addPolynomial(  
    struct Node *p1,  
    struct Node *p2)  
{  
    struct Node *result = NULL;  
  
    while (p1 != NULL && p2 != NULL)  
    {  
        if (p1->exp == p2->exp)  
        {  
            int sum = p1->coeff + p2->coeff;  
  
            if (sum != 0)  
                result = addTerm(  
                    result, sum, p1->exp);  
  
            p1 = p1->next;
```

Polynomial Addition: C Code – Part 2

```
else if (p1->exp > p2->exp)
{
    result = addTerm(
        result,
        p1->coeff,
        p1->exp);

    p1 = p1->next;
}
else
{
    result = addTerm(
        result,
        p2->coeff,
        p2->exp);

    p2 = p2->next;
```

Polynomial Addition: Remaining Terms

```
while (p1 != NULL)
{
    result = addTerm(
        result,
        p1->coeff,
        p1->exp);

    p1 = p1->next;
}

while (p2 != NULL)
{
    result = addTerm(
        result,
        p2->coeff,
        p2->exp);
```

Complete Program: Structure and Node Creation

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int coeff;
7     int exp;
8     struct Node *next;
9 };
10
11 struct Node *createNode(int coeff, int exp)
12 {
13     struct Node *newNode =
14         malloc(sizeof(struct Node));
15
16     if (newNode == NULL)
17         return NULL;
```

Complete Program: Add Term

```
struct Node *addTerm(  
    struct Node *start,  
    int coeff,  
    int exp)  
{  
    struct Node *newNode;  
    struct Node *ptr;  
  
    newNode = createNode(coeff, exp);  
  
    if (start == NULL)  
        return newNode;  
  
    ptr = start;  
  
    while (ptr->next != NULL)  
        ptr = ptr->next;
```

Complete Program: Display Function

```
void displayPolynomial(struct Node *start)
{
    struct Node *ptr = start;

    while (ptr != NULL)
    {
        printf("%dx^%d", ptr->coeff, ptr->exp);

        if (ptr->next != NULL)
            printf(" + ");

        ptr = ptr->next;
    }

    printf("\n");
}
```

Complete Program: Main Function

```
int main()
{
    struct Node *p1 = NULL;
    struct Node *p2 = NULL;
    struct Node *sum = NULL;

    p1 = addTerm(p1, 5, 3);
    p1 = addTerm(p1, 4, 2);
    p1 = addTerm(p1, 2, 1);
    p1 = addTerm(p1, 1, 0);

    p2 = addTerm(p2, 3, 3);
    p2 = addTerm(p2, 2, 2);
    p2 = addTerm(p2, 7, 0);

    printf("Polynomial 1: ");
    displayPolynomial(p1);
}
```

Expected Output

```
Polynomial 1:  
5x^3 + 4x^2 + 2x^1 + 1x^0
```

```
Polynomial 2:  
3x^3 + 2x^2 + 7x^0
```

```
Sum :  
8x^3 + 6x^2 + 2x^1 + 8x^0
```

$$P(x) + Q(x) = 8x^3 + 6x^2 + 2x + 8$$

Example with Missing Powers

Consider:

$$P(x) = 7x^5 + 4x^2 + 3$$

$$Q(x) = 6x^4 + 5x^2 + 2x$$

Represented as:

$$P : (7, 5) \rightarrow (4, 2) \rightarrow (3, 0)$$

$$Q : (6, 4) \rightarrow (5, 2) \rightarrow (2, 1)$$

Addition must correctly handle missing exponents.

Example: Result

Compare terms:

$$7x^5$$

has no matching term.

$$6x^4$$

has no matching term.

For x^2 :

$$4x^2 + 5x^2 = 9x^2$$

Remaining:

$$2x + 3$$

Therefore:

What if Coefficients Cancel?

Consider:

$$5x^3 + 2x$$

and:

$$-5x^3 + 4$$

For exponent 3:

$$5 + (-5) = 0$$

Normally we do not need to store:

$$0x^3$$

Therefore:

Time Complexity

Suppose:

m = number of terms in Polynomial 1

and:

n = number of terms in Polynomial 2

Each list is traversed once.

Therefore:

$$O(m + n)$$

However, if every call to `addTerm()` traverses the result list from the beginning, the practical implementation may incur extra traversal.

Using a tail pointer makes result construction more efficient.

Improving Result Construction

Without a tail pointer:

$$START \rightarrow \dots \rightarrow LAST$$

Every new result term may require finding *LAST* again.

$$START \rightarrow (8, 3) \rightarrow (6, 2) \rightarrow (2, 1)$$

With a tail pointer:

TAIL



always points to the last node.

Therefore appending a new term can take:

TAIL

$$O(1)$$

Common Errors

- Storing polynomial terms in random exponent order.
- Adding coefficients even when exponents are different.
- Moving both pointers when powers are unequal.
- Forgetting remaining terms after one list becomes NULL.
- Treating coefficient as exponent or exponent as coefficient.
- Storing zero-coefficient terms unnecessarily.
- Forgetting to allocate memory for new nodes.

Always Compare Exponents First

Classroom Activity

Represent the following polynomials using linked lists:

$$P(x) = 4x^4 + 3x^2 + 7$$

$$Q(x) = 5x^3 + 6x^2 + 2x + 1$$

Then find:

$$P(x) + Q(x)$$

Draw:

- Polynomial 1 linked list
- Polynomial 2 linked list
- Result linked list

Classroom Activity: Answer

Polynomial 1:

$$(4, 4) \rightarrow (3, 2) \rightarrow (7, 0)$$

Polynomial 2:

$$(5, 3) \rightarrow (6, 2) \rightarrow (2, 1) \rightarrow (1, 0)$$

Result:

$$(4, 4) \rightarrow (5, 3) \rightarrow (9, 2) \rightarrow (2, 1) \rightarrow (8, 0)$$

Therefore:

$$4x^4 + 5x^3 + 9x^2 + 2x + 8$$

Think Before You Code

Suppose:

$$PTR1 \rightarrow (8, 5)$$

and:

$$PTR2 \rightarrow (3, 3)$$

What should we do?

- ➊ Add 8 and 3.
- ➋ Copy (8, 5) to result and move PTR1.
- ➌ Copy (3, 3) and move PTR2.
- ➍ Move both pointers.

Think Before You Code

Suppose:

$$PTR1 \rightarrow (8, 5)$$

and:

$$PTR2 \rightarrow (3, 3)$$

What should we do?

- ➊ Add 8 and 3.
- ➋ Copy (8, 5) to result and move PTR1.
- ➌ Copy (3, 3) and move PTR2.
- ➍ Move both pointers.

Copy (8, 5) and move PTR1

because:

MCQ 1

A node representing a polynomial term must contain:

- ① Only coefficient
- ② Only exponent
- ③ Coefficient, exponent and next pointer
- ④ Only next pointer

MCQ 1

A node representing a polynomial term must contain:

- ① Only coefficient
- ② Only exponent
- ③ Coefficient, exponent and next pointer
- ④ Only next pointer

Coefficient, exponent and next pointer

MCQ 2

The node:

7		3		NEXT
---	--	---	--	------

represents:

- ① $3x^7$
- ② $7x^3$
- ③ $7 + 3x$
- ④ $21x$

MCQ 2

The node:

7		3		NEXT
---	--	---	--	------

represents:

- ① $3x^7$
- ② $7x^3$
- ③ $7 + 3x$
- ④ $21x$

$7x^3$

If:

$$PTR1 - > exp = PTR2 - > exp$$

during polynomial addition, we should:

- ① Compare coefficients
- ② Add coefficients
- ③ Multiply coefficients
- ④ Ignore both terms

If:

$$PTR1 - > exp = PTR2 - > exp$$

during polynomial addition, we should:

- ① Compare coefficients
- ② Add coefficients
- ③ Multiply coefficients
- ④ Ignore both terms

Add coefficients

MCQ 4

If:

$$PTR1 - > exp > PTR2 - > exp$$

then:

- 1 Copy PTR1 term and move PTR1
- 2 Copy PTR2 term and move PTR2
- 3 Add coefficients
- 4 Stop the algorithm

If:

$$PTR1 - > exp > PTR2 - > exp$$

then:

- ① Copy PTR1 term and move PTR1
- ② Copy PTR2 term and move PTR2
- ③ Add coefficients
- ④ Stop the algorithm

Copy PTR1 term and move PTR1

Quick Trace

Given:

$$P(x) = 3x^3 + 5x + 2$$

$$Q(x) = 4x^2 + 2x + 7$$

Find:

$$P(x) + Q(x)$$

Quick Trace

Given:

$$P(x) = 3x^3 + 5x + 2$$

$$Q(x) = 4x^2 + 2x + 7$$

Find:

$$P(x) + Q(x)$$

$$3x^3 + 4x^2 + 7x + 9$$

<i>Coefficient</i>		<i>Exponent</i>		<i>Next</i>
--------------------	--	-----------------	--	-------------

$$6x^3 + 9x^2 + 7x + 1$$



$$(6, 3) \rightarrow (9, 2) \rightarrow (7, 1) \rightarrow (1, 0)$$

Polynomial addition:

Practice Questions

- 1 Represent $5x^4 + 7x^2 + 3$ using a linked list.
- 2 Write the C structure for a polynomial node.
- 3 Write a function to display a polynomial.
- 4 Write an algorithm for adding two polynomials.
- 5 Add:

$$6x^3 + 4x + 2$$

and

$$5x^4 + 3x^3 + 7$$

- 6 Explain the three exponent comparison cases.
- 7 Analyse the complexity of polynomial addition.

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
- ② Reema Thareja, *Data Structures Using C*.
- ③ Horowitz, Sahni and Anderson-Freed, *Fundamentals of Data Structures in C*.

Your Feedback Matters

Please share your feedback about:

- Polynomial node representation
- Linked-list visualization
- Addition algorithm
- Pointer movement
- C programming examples
- Dry-run examples



Scan to Submit Feedback

Thank You

Questions?

COEFF

EXP

NEXT

Polynomial $\rightarrow$ Linked List $\rightarrow$ Addition